

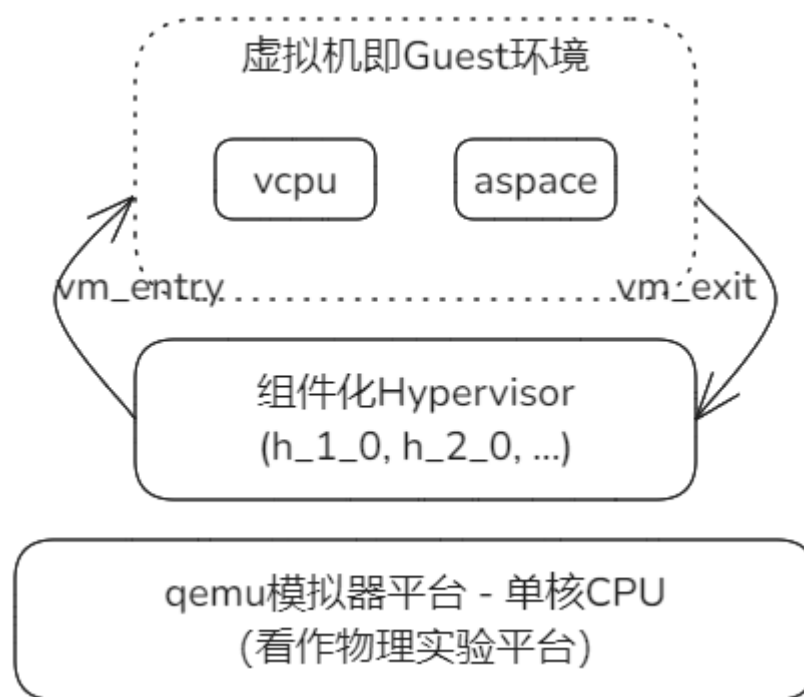
秋冬季训练营三阶段 组件化内核设计与实践(8)

清华大学 & 山东乾云启创
泉城实验室安全操作系统联合创新中心
石磊

2024.11.27

Hypervisor阶段的简化实验模型

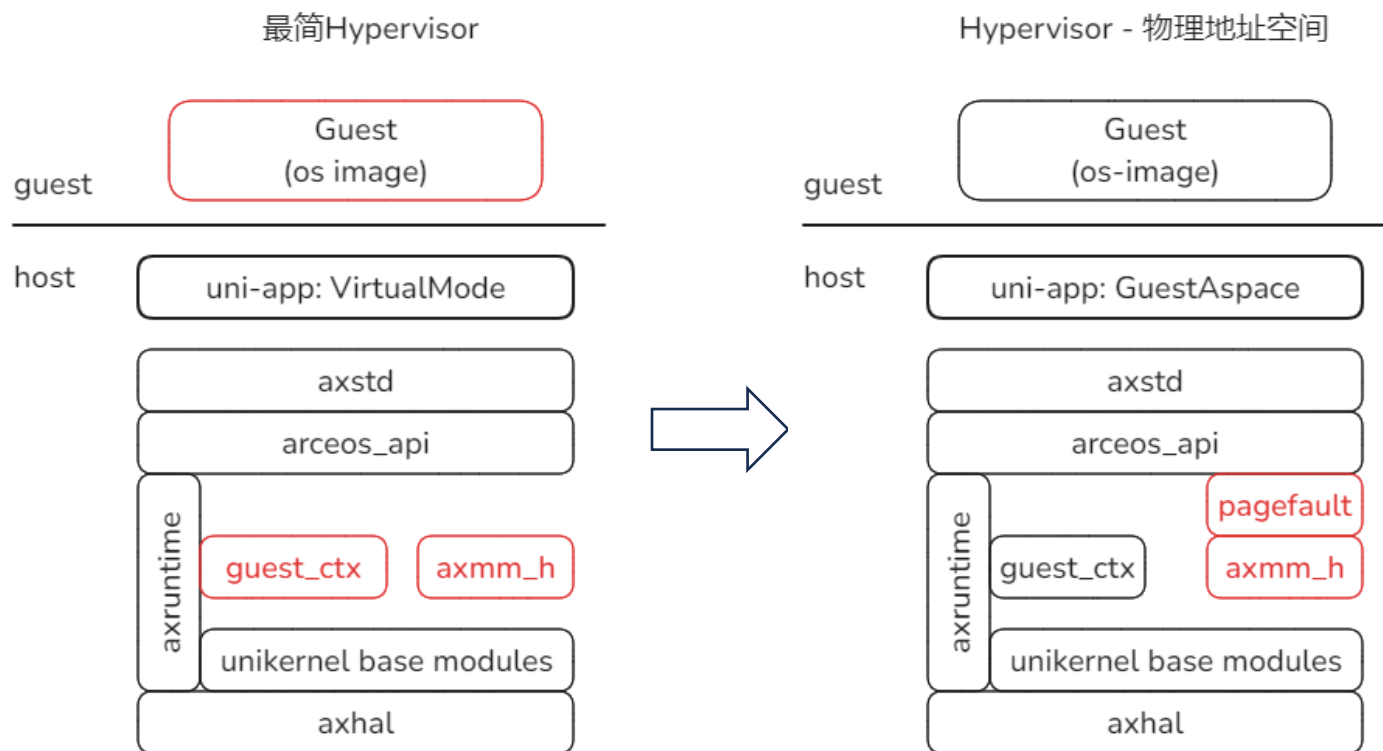
1. 模拟器qemu视为物理环境，仅包含单核CPU
2. Hypervisor和虚拟机仅在单任务环境下运行，无并发
3. 省略VM虚拟机这一级对象，只有虚拟地址空间(aspac)和VCpu(上下文状态)两个对象，虚拟地址空间负责定位虚拟机资源，VCpu负责运行状态的跟踪和切换



在单任务中运行和切换

我们的实验相当于模拟器嵌套虚拟化

H.2.0 GuestAspace



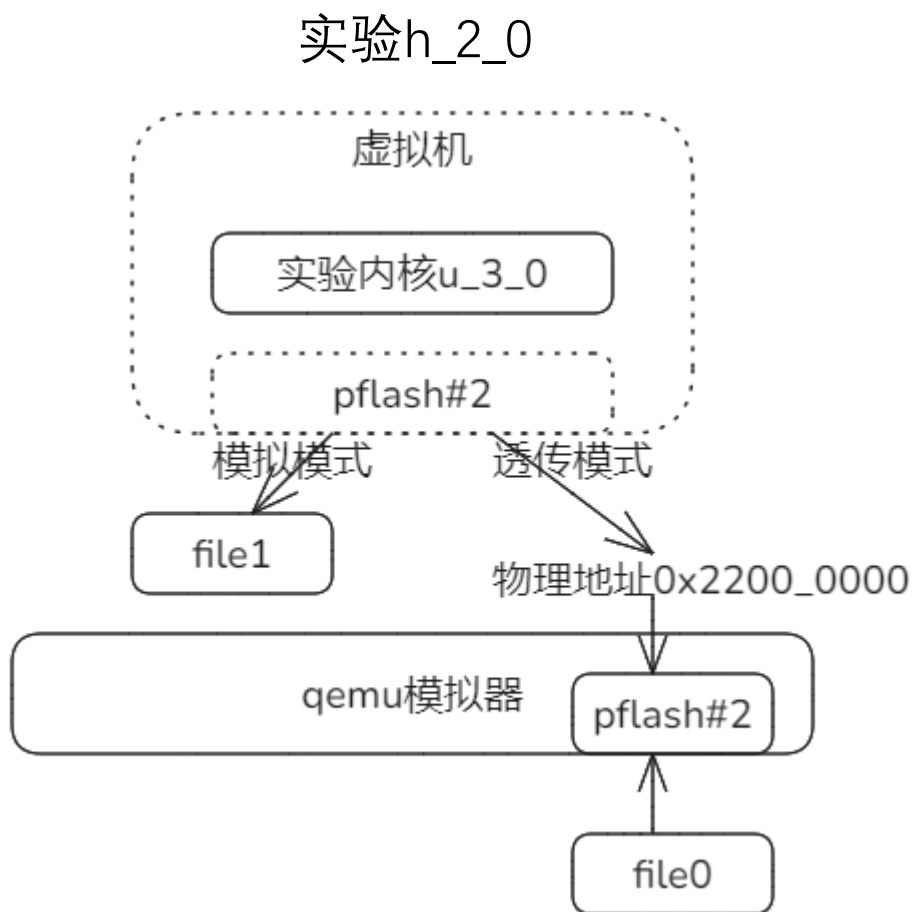
本节目标:

1. 两阶段地址映射的原理和机制
2. 为Guest映射PFlash扩展内存示例

实验命令行: [下面第三行生成u_3_0的示例内核作为实验内核]
make pflash_img
make disk_img (在disk.img不存在时执行)
make A=tour/u_3_0/
./update_disk.sh tour/u_3_0/u_3_0_riscv64-qemu-virt.bin
make run A=tour/h_2_0/ BLK=y

h_2_0实验目标和需要解决的问题

通过实验h_2_0分析两阶段地址映射原理和具体实现。



虚拟机运行的实验内核是第一周的u_3_0：从pflash设备读出数据，验证开头部分。

有两种处理方式：

1. 模拟模式 - 为虚拟机模拟一个pflash，以file1为后备文件。当Guest读该设备时，提供file1文件的内容。
2. 透传模式 - 直接把宿主物理机(即qemu)的pflash透传给虚拟机。

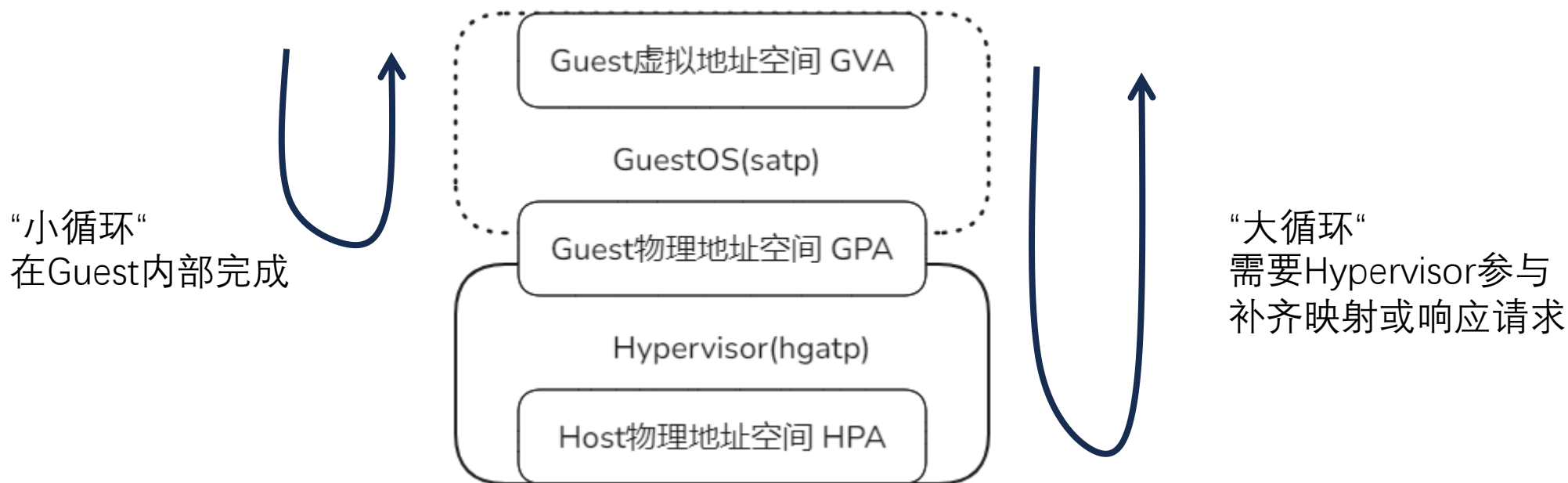
优劣势：模拟模式可为不同虚拟机提供不同的pflash内容，但效率低；透传模式效率高，但是捆绑了设备。

需要解决的问题：

1. 两种模式都涉及两阶段映射的问题。
2. 对两种模式的具体处理方法。

Guest与Host的地址空间关系

Guest是指虚拟机所在的执行环境；Host指Hypervisor所处的执行环境。
Hypervisor负责基于HPA面向Guest映射GPA，基本寄存器是hvatp；
Guest认为看到的GPA是“实际”的物理空间，它基于satp映射内部的GVA虚拟空间。



体系结构riscv64对Guest地址映射支持

RISC-V G Stage

- vsatp: Virtual Supervisor Address Translation and Protection Register, 用于第一阶段页表翻译。
- hgatp: Hypervisor Guest Address Translation and Protection Register, 用于第二阶段页表翻译。
- GVA $\rightarrow$ (vsatp) $\rightarrow$ GPA $\rightarrow$ (hgatp) $\rightarrow$ HPA

hgatp相对于satp在根页表级扩展了2位，总数达到41位

原模式: 直接地址映射

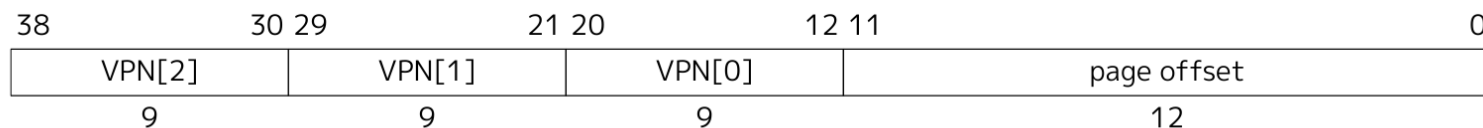
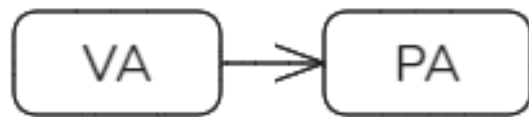


Figure 60. Sv39 virtual address.

虚拟化: 两阶段地址映射

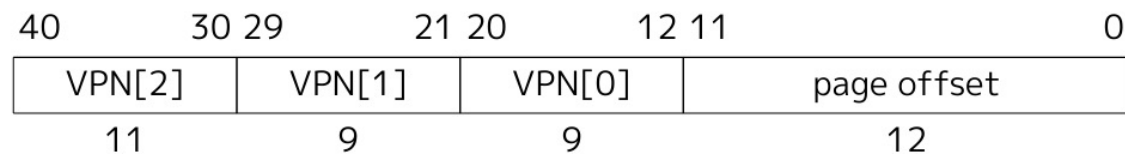
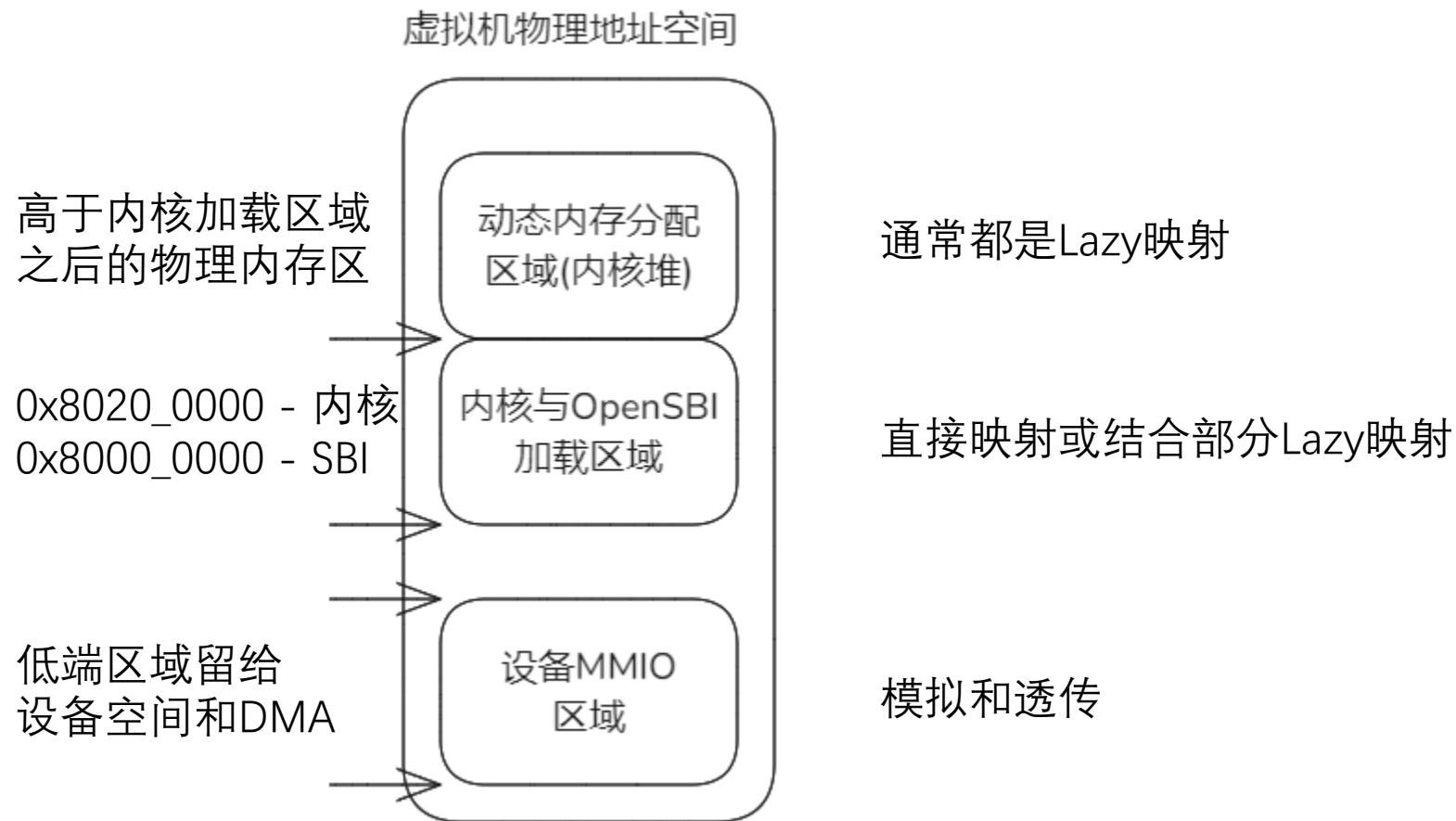


Figure 108. Sv39x4 virtual address (guest physical address).

虚拟机物理地址空间布局

为虚拟机呈现与所模拟的物理平台相同的物理地址空间布局。

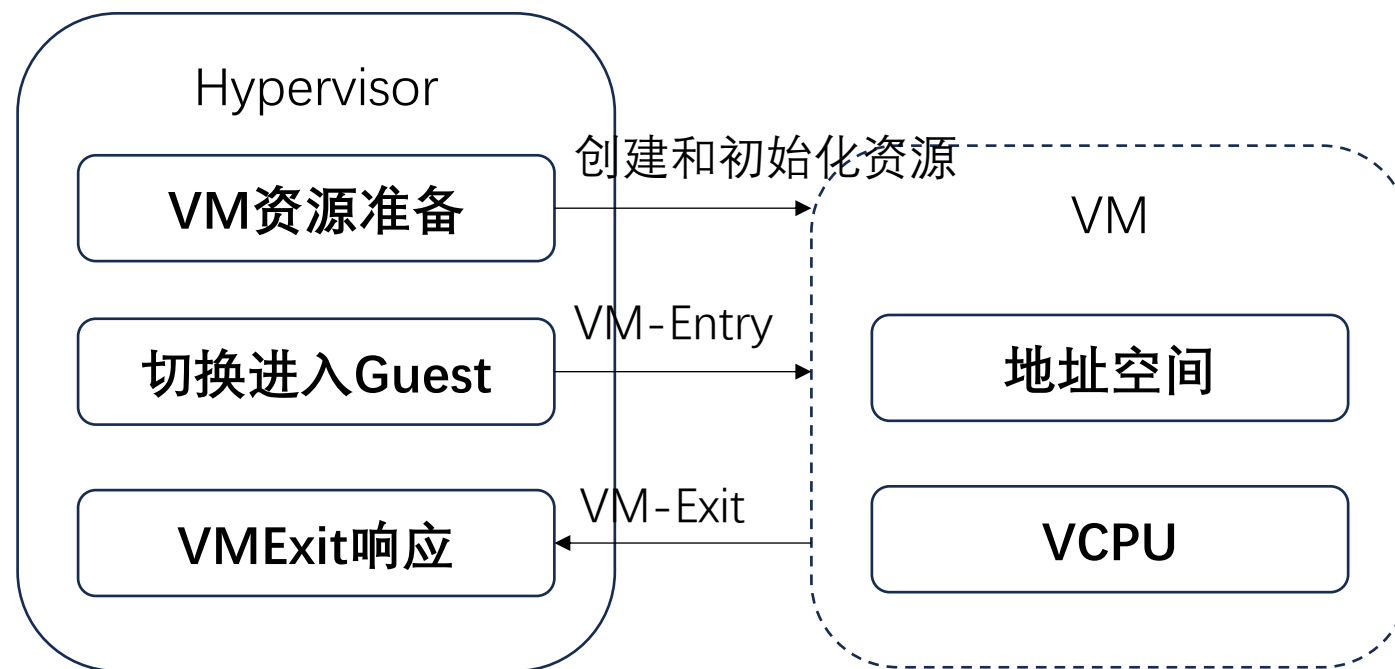


实验H_2_0的结构

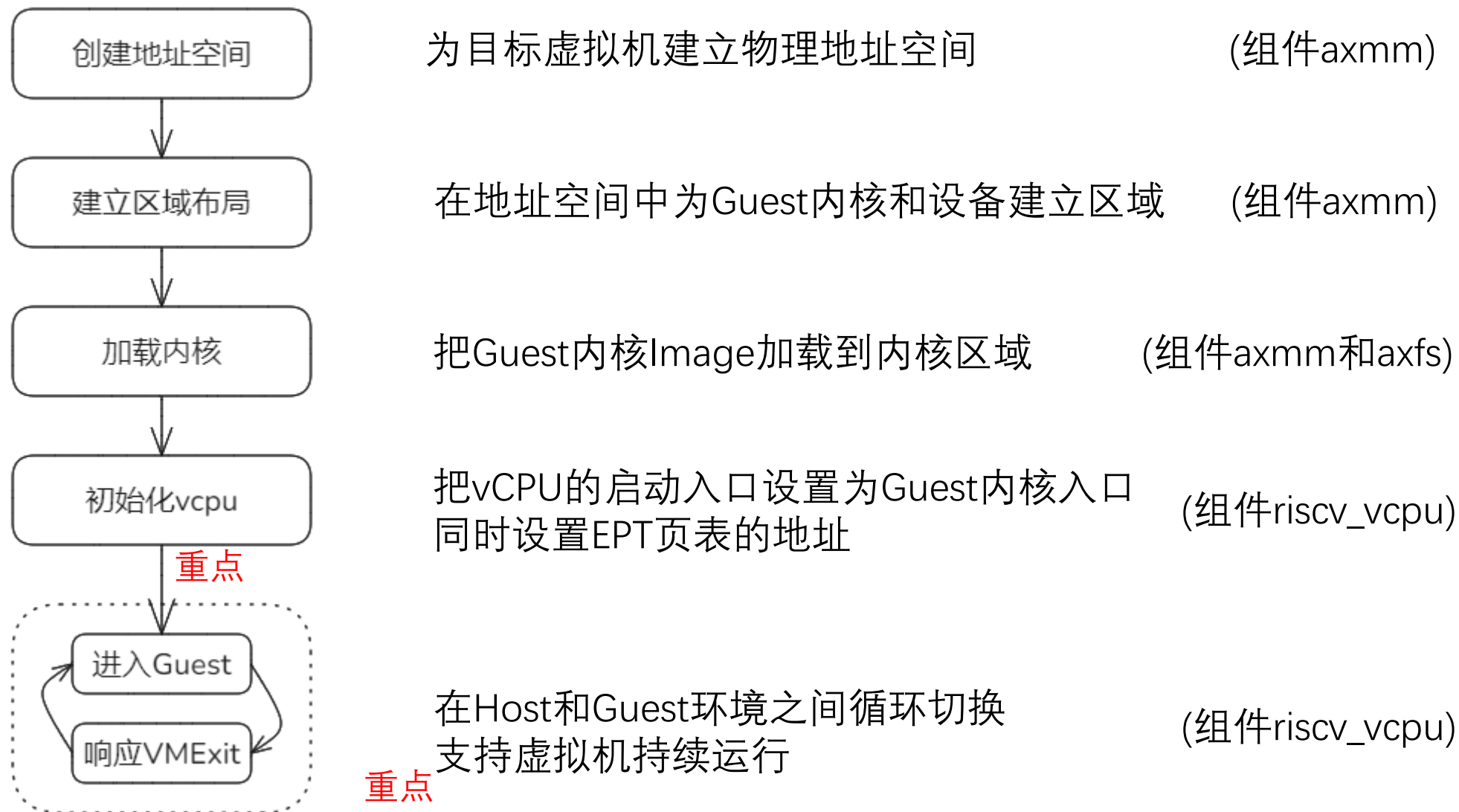
按照简化实验模型，实验只有一个VM，所以忽略该对象。

Hypervisor的主逻辑包含三个部分：

1. 准备VM的资源：VM地址空间和单个vCPU
2. 切换进入Guest的代码
3. 响应VMExit各种原因的代码



实验H_2_0的实现流程



VCPU的准备

VCPU的实现与体系结构紧密相关，但操作上类似。

在准备进入虚拟化模式之前，设置GuestOS内核启动入口地址和Guest页表地址。

```
// Create VCpus.  
let mut arch_vcpu = RISCVVCpu::init();  
  
// Setup VCpus.  
info!("bsp_entry: {:#x}; ept: {:#x}", VM_ENTRY, aspace.page_table_root());  
arch_vcpu.set_entry(VM_ENTRY.into()).unwrap();  
arch_vcpu.set_ept_root(aspace.page_table_root()).unwrap();
```

VM_ENTRY入口地址就是0x8020_0000

modules/riscv_vcpu/src/vcpu.rs

```
pub fn set_entry(&mut self, entry: GuestPhysAddr)  
{  
    let regs = &mut self.regs;  
    regs.guest_regs.sepc = entry.as_usize();  
    Ok(())  
}
```

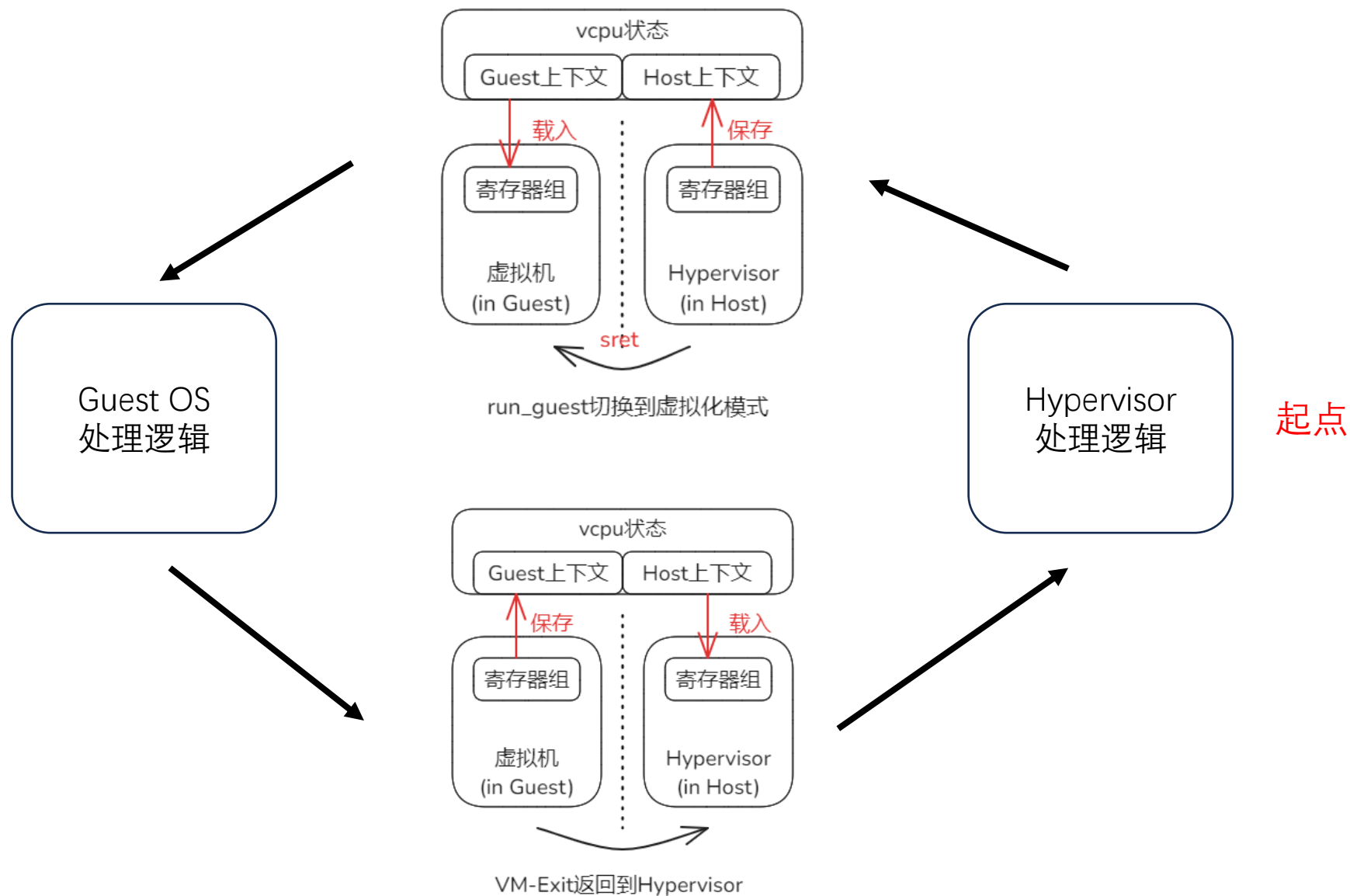
当通过run_guest进入Guest虚拟机前，设置的regs.guest_regs.spec会进一步被写入sepc寄存器。那么进入Guest之后，就从该地址启动内核。

modules/riscv_vcpu/src/vcpu.rs

```
pub fn set_ept_root(&mut self, ept_root: HostPhysAddr) -> Ax  
{  
    self.regs.virtual_hs_csrs.hgatp =  
        8usize << 60 | usize::from(ept_root) >> 12;  
    unsafe {  
        core::arch::asm!(  
            "csrw hgatp, {hgatp}",  
            hgatp = in(reg) self.regs.virtual_hs_csrs.hgatp,  
        );  
        core::arch::riscv64::hfence_gvma_all();  
    }  
    Ok(())  
}
```

向hgatp设置模式和根页表的页帧

在Host与Guest环境之间切换的框架

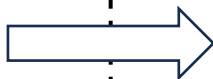


在Host与Guest环境之间切换的框架

框架达到的目的：无论是首次还是将来进入Guest，都经由同一个入口vcpu_run。

tour/h_2_0/src/main.rs

```
loop {  
    match vcpu_run(&mut arch_vcpu) {  
        Ok(exit_reason) => match exit_reason {  
            ... ..  
        },  
        Err(err) => {  
            panic!("run VCpu get error {:?}", err);  
        }  
    }  
}
```



modules/riscv_vcpu/src/vcpu.rs

```
fn run(&mut self) -> AxResult<...> {  
    let regs = &mut self.regs;  
    unsafe { _run_guest(regs); }  
    self.vmexit_handler()  
}
```

无限循环执行vcpu_run和处理vm_exit情况：

- vcpu_run在关中断情况下调用riscv_vcpu.run()
- 本层处理体系结构无关的exit_reason

riscv_vcpu的run包含两部分：

- **_run_guest()**负责切换进入Guest环境
- vmexit_handler()处理体系结构相关的exit_reason

虚拟模式切换的关键run_guest

run_guest负责进入Guest，定义在modules/riscv_vcpu/src/guest.S
详细信息可以参考其注释，总体流程主要分为5步：

```
run_guest:
    /* Save hypervisor state */

    /* Save hypervisor GPRs (ex
sd    ra, ({hyp_ra})(a0)
sd    gp, ({hyp_gp})(a0)
sd    tp, ({hyp_tp})(a0)
```

第1步：保存Host状态，
以备从Guest返回时恢复

```
ld    t1, ({guest_sepc})(a0)
csw    sepc, t1
```

第2步：设置sepc等寄存器，
指示Guest从断点处继续
(首次设置是0x8020_0000)

```
la    t1, _guest_exit
csrrw t1, stvec, t1
sd    t1, ({hyp_stvec})(a0)
```

第3步：设置stvec寄存器，让
Guest返回时进入_guest_exit

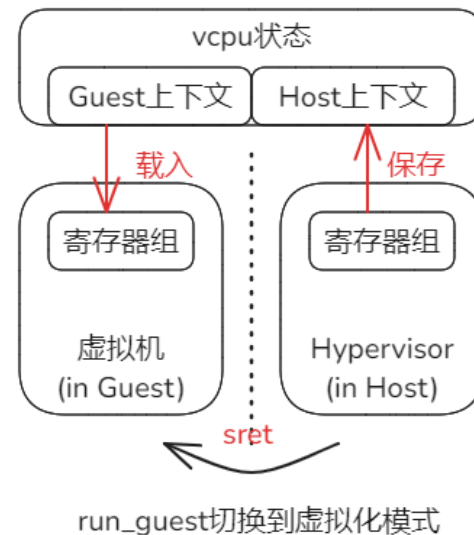
```
ld    ra, ({guest_ra})(a0)
ld    gp, ({guest_gp})(a0)
ld    tp, ({guest_tp})(a0)
```

第4步：恢复Guest的通
用寄存器组状态

```
sret
```

第5步：之前设置hstatus指示是从
Guest进入，执行sret切换到Guest

除了第3步之外对应下图



第3步为Guest退出时
进入_guest_exit作准备

虚拟模式切换的关键guest_exit

由于run_guest进入Guest时设置stvec为guest_exit，所以退出Guest时进入此处。

modules/riscv_vcpu/src/guest.S

```
.align 2
_guest_exit:
    /* Pull GuestInfo out of
       csrrw a0, sscratch, a0

    /* Save guest GPRs. */
    sd    ra, ({guest_ra})(a0)
```

```
/* Restore hypervisor GPRs. */
ld    ra, ({hyp_ra})(a0)
ld    gp, ({hyp_gp})(a0)
ld    tp, ({hyp_tp})(a0)
ld    s0, ({hyp_s0})(a0)
```

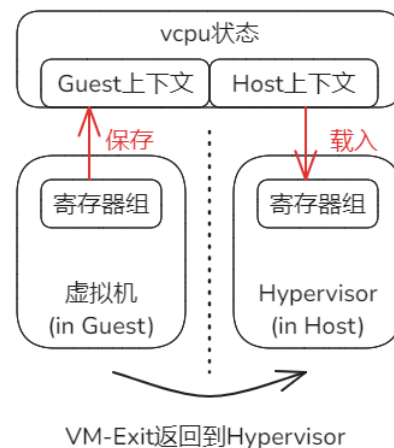
```
ret
```

_guest_exit的流程基本是
_run_guest流程的逆过程。
此处略。

需要注意以下两点：

第1点：hyp_ra保存着进入Guest前run_guest函数的返回地址，此处恢复ra。

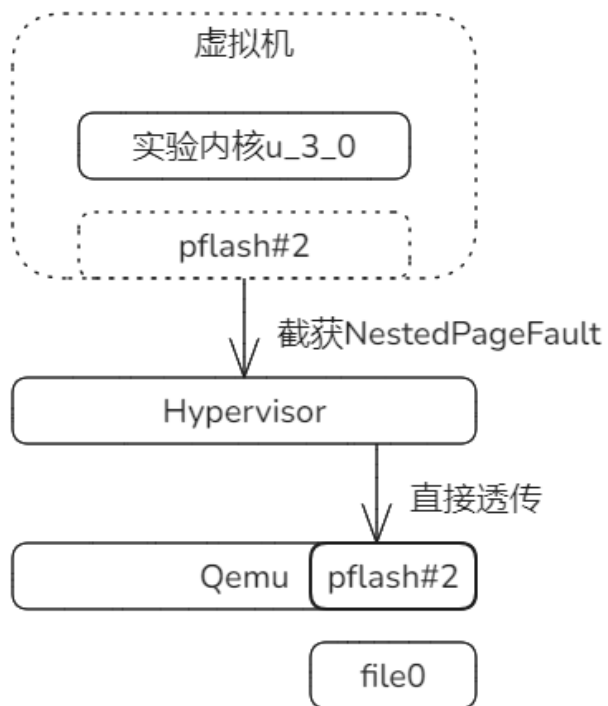
第2点：此处ret，基于ra
会从run_guest函数返回。



```
fn run(&mut self) -> AxResult<...> {
    let regs = &mut self.regs;
    unsafe { _run_guest(regs); }
    self.vmexit_handler()
}
```

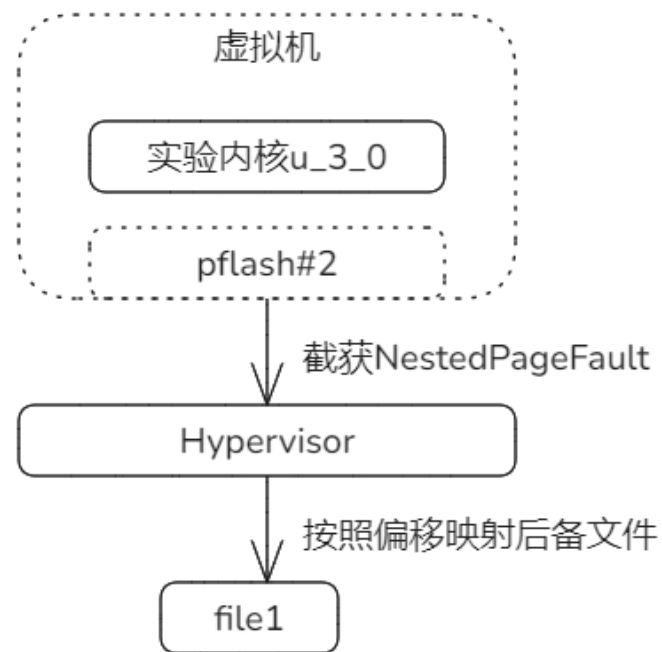
进入Guest，退出
返回时到下行

设备模拟和透传(默认)两种模式的实现



用`map_linear`直接映射
qemu(宿主平台)的pflash#2的地址区域

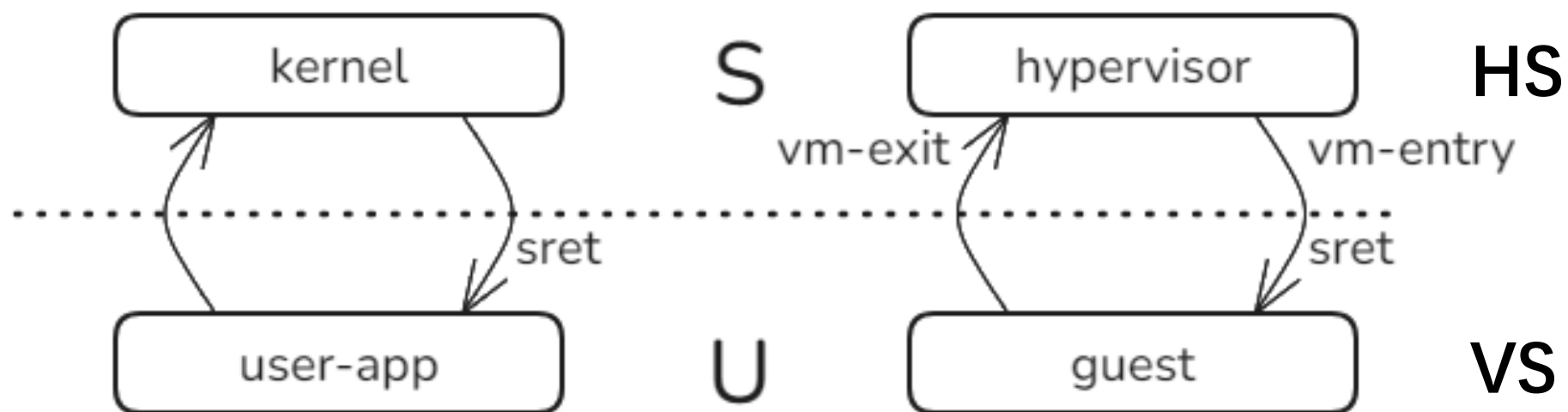
```
// Passthrough-Mode
let _ = aspace.map_linear(
    addr, addr.as_usize().into(), 4096, mapping_flags);
```



用`map_alloc`申请页帧完成映射
从file1中加载内容填充页帧

```
// Emulator-Mode
// Pretend to load file to fill buffer.
let buf = "pfl1d";
aspace.map_alloc(addr, 4096, mapping_flags, true);
aspace.write(addr, buf.as_bytes());
```

hypervisor与宏内核对比



课后练习

在h_2_0的基础上，把pflash设备模拟方式补充完整。目前实现并未真正加载外部文件，要求让Hypervisor能够从disk.img中读pflash后备文件填充映射页帧。

预备：

1. 创建一个单独分支进行练习。
2. 删除Passthrough-Mode的代码，放开Emulator-Mode的代码。如图。

```
// Passthrough-Mode
//let _ = aspace.map_linear(addr, addr.as_usize().

// Emulator-Mode
// Pretend to load file to fill buffer.
let buf = "pflid";
aspace.map_alloc(addr, 4096, mapping_flags, true);
aspace.write(addr, buf.as_bytes());
```

3. 编译执行h_2_0，输出与之前相同。

要求：Hypervisor能够从disk.img中读pflash后备文件填充映射页帧。使得测例通过。

提示：

1. 可以利用update_disk.sh把制造的pflash后备文件写入disk.img
2. 注意：我们的实验类似嵌套虚拟化，Hypervisor在qemu中运行，无法直接读真实物理机文件。